TURING-AGENT · AUTONOMOUS QA SYSTEM · POWERED BY TINYFISH

Bug reports in.
Evidence out.

An autonomous GitHub QA maintainer. It validates every bug report, correlates it with the repository history using TinyFish, and ships maintainer-approved fixes as pull requests, on GitHub and Jira.

1 Report arrives

GitHub Discussion or Jira ticket, acknowledged instantly

2 TinyFish correlation

Related issues, PRs and commits scraped and scoped

3 Evidence, then a PR

Maintainer types approved; Turing commits and opens a pull request

github.com/YASHK-arch/Turing-agent · Backend: turing-agent-backend.onrender.com

OVERVIEW

Eight capabilities, one bug lifecycle

Runs on GitHub and Jira webhooks, an n8n orchestration pipeline and a FastAPI backend.

1 Instant acknowledgment

Reacts with an EYES emoji on every new discussion immediately.

2 Zero-prompt validation

A maintainer-style review of the bug report, with no human prompt needed.

3 Cross-issue intelligence

TinyFish Fetch API scrapes issues, PRs, commits and discussions to correlate components and scope the fix.

4 Threaded reporting

The correlation report is posted as a nested reply under the validation comment.

5 Autonomous issue and PR

On maintainer **approved**: fetch files in parallel, generate the fix with Groq, commit via the Git Data API and open a PR with co-author credit.

6 Collaborator auto-accept

Accepts repository invitations sent to the Turing-agent bot account.

7 Jira integration

Posts INTERNAL NOTES, handles RW / approved / Hi Turing, and creates GitHub Issues labelled source: Jira.

8 Git regression intelligence

Git DAG dashboard with defect pinning and culprit commit detection. Planned for a later release.

FUTURE

SYSTEM ARCHITECTURE

Four layers, one clear flow

GitHub and Jira events are routed by n8n to a FastAPI backend that calls TinyFish, Groq, GitHub and Jira.

EVENT SOURCES

GITHUB

- New discussion opened
- @Turing mention
- Maintainer: approved
- Collaborator invitation

JIRA

- New ticket / commands

N8N ORCHESTRATION

- Webhook intake
- Route by Event switch

BRANCHES

- New Bug Report
- Turing Mention
- Maintainer Approved
- RW Command

FASTAPI BACKEND

- `/triage-discussion`
- `/chat-turing`
- `/resolve-issue`
- `/consolidate-discussion`
- `/accept-turing-invite`
- `/jira/triage`
- `/jira/comment`

EXTERNAL SERVICES

- TinyFish Web Fetch API
- Groq LLM (openai/gpt-oss-120b)
- GitHub REST and GraphQL API
- Jira REST API

FUTURE **Future:** React dashboard (Git DAG, incident pinning, culprit detection) on top of the same backend.

PIPELINE 1 AND 2

Validate and correlate the moment a report lands

Runs once, when a discussion is created. Later comments never re-trigger the validation template.

Zero-prompt validation

1 Acknowledge

An EYES reaction fires immediately on the new discussion.

2 Validate

POST /triage-discussion writes a warm, professional review: confirms the bug is valid, states the root-cause hypothesis and tells the author TinyFish is gathering context.

3 Post

The review becomes the primary response in the thread.

Single-validation rule. Validation and the TinyFish report fire only when the discussion opens.

TinyFish cross-issue correlator

1 Scrape

Fetch API reads open and closed issues and the discussions page.

2 Deep-scrape

Top overlaps (e.g. #147, #131) are scraped for linked PRs, commit hashes and maintainer comments.

3 Correlate and reply

Groq maps earlier resolutions to the new issue; the report is nested under the validation via replyToId.

THE REPORT CONTAINS

- Related issues table: issue, state, component, actions taken
- Cross-component impact and resolution mapping
- "Ready for Work" scope: files, solution pattern, acceptance criteria

PIPELINE 3

Intent-driven live Q&A

Triggered by comments that mention Turing or @Turing-agent. A Groq classifier picks one of three handlers.

FIND_RELATED_ISSUES

When: Asks about related issues, duplicates, previous PRs or conflicting changes.

Turing: TinyFish Fetch scrapes issues, PRs and discussions and returns clickable links.

ANALYZE_PROPOSAL

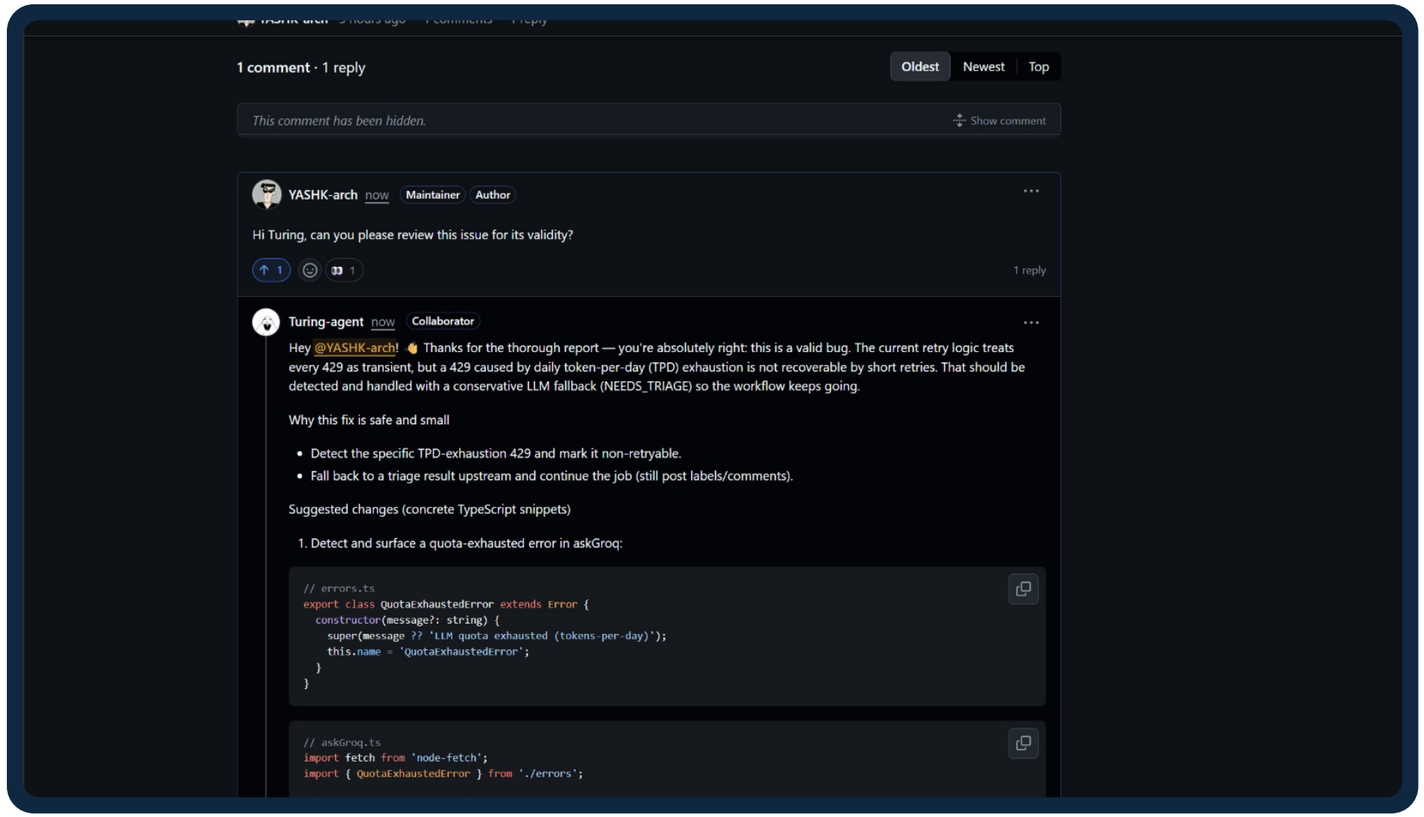
When: Rejects a suggestion or proposes an alternative approach.

Turing: Critique of strengths, risks and trade-offs, with a concrete patch.

GENERAL_QA

When: Codebase questions, debugging tips, project explanations.

Turing: A direct, friendly reply grounded in the code.



Sample thread. The maintainer asks Turing to review an issue; Turing replies in-thread with a verdict, the root cause, why the fix is safe and small, and concrete code snippets to review.

PIPELINE 4

From maintainer approval to pull request

One word from a maintainer starts the whole fix workflow.

1. **Create issue**
   Opens an official GitHub issue labelled **ready-for-work**, linked back to the discussion.

2. **Post progress checklist**
   An in-thread checklist is ticked off live: intake, analyzing, coding, committed.

3. **Generate the fix**
   Fetches repo files in parallel (8 workers, under 1.5 s). Groq returns complete modified files, a commit message and a summary.

4. **Atomic commit**
   Creates blobs and a tree through the Git Data API, commits with *Co-authored-by* credit and pushes **issue-<number>**.

5. **Open the PR**
   PR includes the problem, AI fix summary, file diffs and issue references. The checklist is finalized with the PR link.

TRIGGER AND GUARD

A comment matching **/approved/i** only counts when its author is an **OWNER**, **MEMBER** or **COLLABORATOR**.

OUTPUT

A real commit and a pull request with co-author credit, plus a progress comment that updates in place with zero comment pollution.

n8n path: Create Issue, Get Base Commit, Call Resolve Issue API, Build Tree, Create Commit, Create Branch, Open PR.

PIPELINES 5 TO 7

Beyond the fix: consolidate, integrate, onboard

RW: raise work

Maintainer comments **RW**

- Reads the whole thread, ignoring bot checklists.
- Groq consolidates it into a clean issue spec: motivation, agreed implementation, file scope, acceptance criteria.
- Patches the existing GitHub issue with a new title, body and labels.
- Capped at 2,500 characters for LLM safety.

Jira integration

POST /jira/triage and /jira/comment

- Validation and TinyFish correlation are posted as an INTERNAL NOTE.
- **RW** creates a GitHub Issue labelled source: Jira and replies to the customer.
- **approved** runs the automated PR fix workflow.
- **Hi Turing, ...** goes through the intent layer.

Collaborator auto-accept

GET / POST /accept-turing-invite

- Lists pending repository invitations for the bot.
- Accepts each one with the Turing-agent token.
- Runs periodically and on webhook, so onboarding takes one step.

Give maintainers their focus back

Less time proving bugs. More time fixing them.

YK

Yash Kumar

Team Lead · Backend & TinyFish

Owns the Turing-agent FastAPI service, TinyFish agent runs and the GitHub repository.

OR

Om Rajput

Automation & Triage

Builds the n8n workflows, GitHub comment flow and Groq triage prompts.

AJ

Akshat Jain

Research, QA & Demo

Writes bug-report scenarios and test cases, and prepares the demo and submission docs.

Project repository **github.com/YASHK-arch/Turing-agent**